

プログラミング言語論

第11回 論理型プログラミング

篠埜 功

論理型プログラミング (logic programming)

- 自動定理証明がもとになっている
- 1階述語論理の構文が用いられる
- Prolog (1973年) --- 最初は自然言語処理に用いられた
- Robert Kowalski: “algorithm=logic+control”
- Alain Colmerauerと彼のteamが自然言語処理用のプログラミング言語を開発しており、Kowalskiと交流してPrologの開発に繋がった。

Algorithm = logic + control

- Logic --- アルゴリズムが何をするかを示す事実と規則(プログラマーが記述する)
- Control --- アルゴリズムがどのように実装されるか(言語が提供する)

Prologにはさまざまな方言(dialect)が存在する。
代表的なものはEdinburgh Prolog。Edinburgh Prologが
PrologのISO規格に影響を与えている。

各方言 D の間の構文以外の違いは次の式で表される:

$$\text{algorithm}_D = \text{logic} + \text{control}_D$$

(参考文献) R. A. Kowalski, "Algorithm = Logic + Control".

Communication of the ACM, 22(7), pp. 424-436, 1979

論理型プログラミングの概念

- 関数 (function) ではなく関係 (relation) を用いる
- n 項関係は関係は m 行 n 列 (行は無限にあるかもしれない) のテーブルで表される
 - $(a_1, a_2, \dots, a_n)$ がテーブル内のどこかの行の場合、 $a_1, a_2, \dots, a_n$ はそのテーブルが表している関係にある。

例: append

X	Y	Z
[]	[]	[]
[a]	[]	[a]
[a]	[b]	[a,b]
[a,b]	[c,d]	[a,b,c,d]
...	...	...

関係appendは「XとYを連結したリストがZ」を満たす組
(X,Y,Z)の集合(それを表の形にしたものが上記の表)
関係は述語でもある:「与えられた組は関係の要素か」
(例) $([a],[b],[a,b]) \in \text{append}$, $([a],[b],[]) \notin \text{append}$

ホーン節(Horn clause)

関係を、 $P :- Q_1, Q_2, \dots, Q_k$. のような形の規則により記述する($k \geq 0$)。これは以下のような論理式に対応する。

$P \text{ if } Q_1 \text{ and } Q_2 \text{ and } \dots \text{ and } Q_k . \ (k \geq 0)$

$Q_1, Q_2, \dots, Q_k$ が成り立つなら P が成り立つという意味 (declarative interpretation)だが、 P を成立させるには $Q_1, Q_2, \dots, Q_k$ を成立させればよいというように考える (procedural interpretation)。このような規則をホーン節 (Horn clause)と呼ぶ。 k が0のときは前提無しで成り立つ事実を表し、 $:-$ を省略して P . のように記述する。

(例) 関係appendは2つの規則で記述される。

append ([], Y, Y).

append ([H|X], Y, [H|Z]) :- append (X,Y,Z).

(参考文献) A. Horn, "On sentences which are true of direct unions of algebras". *Journal of Symbolic Logic*, Vol. 16, pp. 14-21, 1951.

Query

?- **append([a,b],[c,d],[a,b,c,d]).**

yes

?- **append([a,b],[c,d],Z).**

Z=[a,b,c,d]

yes

?- **append([a,b],Y,[a,b,c,d]).**

Y=[c,d]

yes

?- **append(X,[c,d],[a,b,c,d]).**

X=[a,b]

yes

?- **append(X,[d,c],[a,b,c,d]).**

no

論理型プログラミングにおいては(変数を含む)queryに対する答えを見つけるのが計算。

Term

Simple term:

数

大文字で始まる変数

アトム(atom)

(例) 0 1972 X Source lisp algol60

Compound term:

アトムの後にtermの列を括弧で囲んだもの

(例) link(bcpl, c)

中値記法が使える場合もある。

(例) =(X,Y)は $X=Y$ と書いてよい。

_は特別な変数でplaceholder。

Fact, rule, queryの構文 (Edinburgh Prologの場合)

$\langle fact \rangle ::= \langle term \rangle .$

$\langle rule \rangle ::= \langle term \rangle :- \langle terms \rangle .$

$\langle query \rangle ::= \langle terms \rangle .$

$\langle term \rangle ::= \langle number \rangle \mid \langle atom \rangle \mid \langle variable \rangle$
 $\mid \langle atom \rangle (\langle terms \rangle)$

$\langle terms \rangle ::= \langle term \rangle \mid \langle term \rangle, \langle terms \rangle$

Fact, ruleの例

link(fortran, algol60).

link(algol60, cpl).

link(cpl, bcpl).

link(bcpl, c).

link(c, cplusplus).

link(algol60, simula67).

link(simula67, cplusplus).

link(simula67, smalltalk80).

path(L,L).

path(L,M) :- link(L,X), path(X,M) .

Existential queries

Query $\langle term \rangle_1, \langle term \rangle_2, \dots, \langle term \rangle_k$.

は、 $\langle term \rangle_1$ and $\langle term \rangle_2$ and ... and $\langle term \rangle_k$?

という疑似コードに対応する。Queryはgoalとも呼ばれる。Query中の各termはsubgoalとも呼ばれる。

(例) ?- link(cpl, bcpl), link(bcpl, c).

yes

?- link(algol60, L), link(L, M).

L=cpl

M=bcpl

link(algol60,L) と link(L,M)を満たす L,Mはあるか？
--

ここでreturnキーを押すとyesと表示されて終了するが、;を打つと別の解が表示される(かあるいは解が見つからない場合はnoが表示される)。

(続き)

?- link(algol60, L), link(L, M).

L=cpl

M=bcpl ;

L=simula67

M=cplusplus ;

L=simula67

M=smalltalk80

yes

次の解がないことが分かった場合はこのように;の入力を待たずにyesが表示される。

Universal facts and rules

Rule $\langle term \rangle \text{ :- } \langle term \rangle_1, \langle term \rangle_2, \dots, \langle term \rangle_k .$

は、 $\langle term \rangle$ if $\langle term \rangle_1$ and $\langle term \rangle_2$ and ... and $\langle term \rangle_k$?
という疑似コードに対応する。Ruleの:-の左側はhead、:-の右側はconditionあるいはbodyと呼ばれる。Factはruleの特別な場合で、conditionがなくheadだけのruleである。

$path(L, L).$

$path(L, M) \text{ :- } link(L, X), path(X, M) .$

は $path$ という関係を定義している。

$path(L, L)$ は、全ての L に対し、 $path(L, L)$ が成り立つということを表す。 $path(L, M) \text{ :- } link(L, X), path(X, M) .$ は、すべての L と M に対し、もし $link(L, X)$ と $path(X, M)$ が成り立つような X が存在するなら、 $path(L, M)$ が成り立つということを表す。

Negation as failure

Prologは、queryを満たすのに失敗したときに`no`と答える。

```
?- link(lisp,scheme) .
```

`no`

not演算子(`¬`)もnegation as failureであり、`¬(P)`は`P`を示すのに失敗したらtrueとして扱われる。

(続き)

?- *link(L,N), link(M,N) .*

L=fortran

M=fortran

N=algol60

?- *link(L,N), link(M,N), ✕+(L=M) .*

L=c

M=simula67

N=cplusplus ;

L=simula67

M=c

N=cplusplus ;

no

?- *✕+(L=M), link(L,N), link(M,N) .*

no

単一化(unification)

- 2つのtermを等しくするような変数への最も一般的な置換(most general unifier)を求めること
- Unificationは規則の適用ができるかどうかを判定するために行われる。
- Unificationを行う組み込みの述語として=がある。

(例) ?- $f(X,b) = f(a,Y)$.

$X=a$

$Y=b$

(参考文献1) John A. Robinson. “A machine-oriented logic based on the resolution principle”. *Journal of the ACM*, 12(1):23–41, 1965.

(参考文献2) Alberto Martelli and Ugo Montanari, “An efficient unification algorithm”, *ACM TOPLAS* 4(2), pp. 258-282, 1982.

(参考) 片方のみに変数がある場合はパターンマッチング(pattern matching)。

instance

term T の中の変数を何らかのtermで置き換えて得られるtermをterm T のinstanceという。(同じ変数に対しては同じtermで置き換えなければならない。)

(例1) $f(a,b)$ は $f(X,b)$ のinstance。

(例2) $f(a,b)$ は $f(a,Y)$ のinstance。

(例3) $g(a,a)$ は $g(X,X)$ のinstance。

(例4) $g(h(b),h(b))$ は $g(X,X)$ のinstance。

(例5) $g(a,b)$ は $g(X,X)$ のinstanceではない。

2つのterm T_1, T_2 は共通のinstance T を持つとき単一化(unify)できるという。

(参考) 単一化は関数型言語の型推論においても用いられる。

Occurs check

- 変数Xとterm Tを単一化する際に、Tの中にXがあるかどうかを確認することをoccurs checkという。

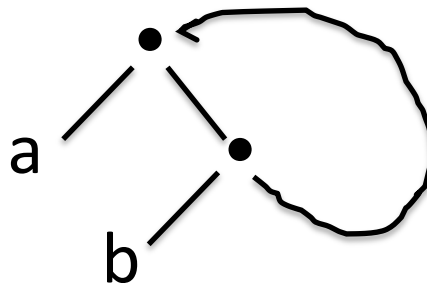
?- **append([], E, [a,b|E]).**

$E = [a,b,a,b,a,b,a,b,a,b,\dots]$

append([], Y, Y)と単一化が成功するためには、YがEと[a,b|E]の両方と単一化が成功しなければならない。Eを[a,b|E]で置き換えようとすると、

$E = [a,b|E] = [a,b,a,b|E] = [a,b,a,b,a,b|E] = \dots$

のようになる。GNU prologなど、cyclic termを構築するものもある。



算術演算

- =演算子 --- term1=term2の形で用いられ、term1とterm2の単一化を行う

?- $X=2+3$.

$X=2+3$

- is演算子 --- term is 式の形で用いられ、式の評価結果とtermの単一化を行う

?- X is $2+3$.

$X=5$

?- X is $2+3$, $X=5$.

$X=5$

?- X is $2+3$, $X=2+3$.

no

Prolog探索木

- 各ノードはゴール(goal)を表す
- 各ノードはそのノードの一番左のサブゴールに適用できるルールの数だけ子供を持つ
- 各ノードの子供の順番は規則の順番と同じ
- Prologの計算は、Prolog探索木を深さ優先で探索することにより行われ、ゴールが空のノードに到達する度に答えを出力する
- Backtrack(バックトラック)が行われる

例

Prolog探索木

a(1) :- b.

a(2) :- e.

b :- c.

b :- d.

c :- fail.

d.

e.

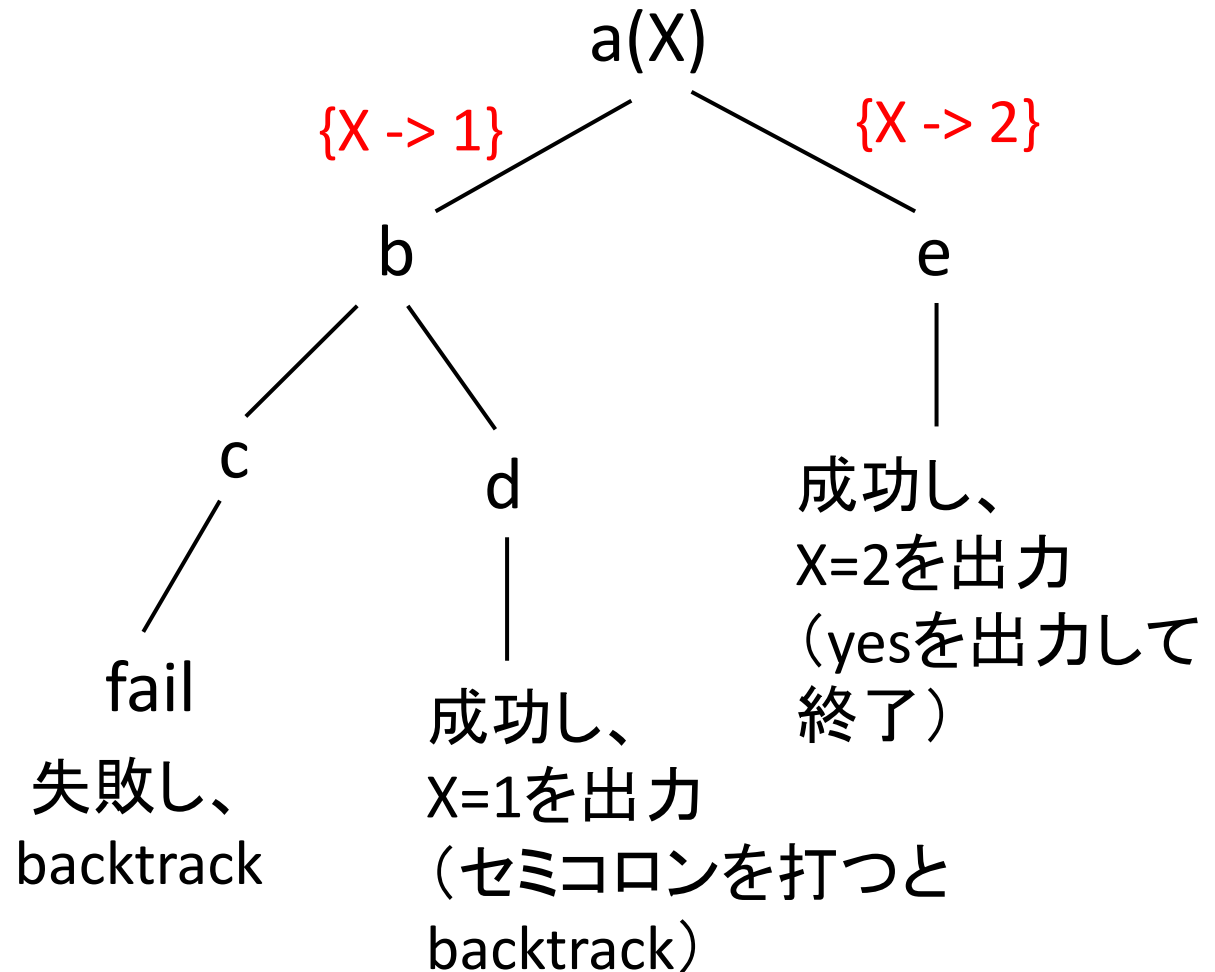
(failは常に失敗する述語)

?- a(X).

X=1;

X=2

yes



Cut

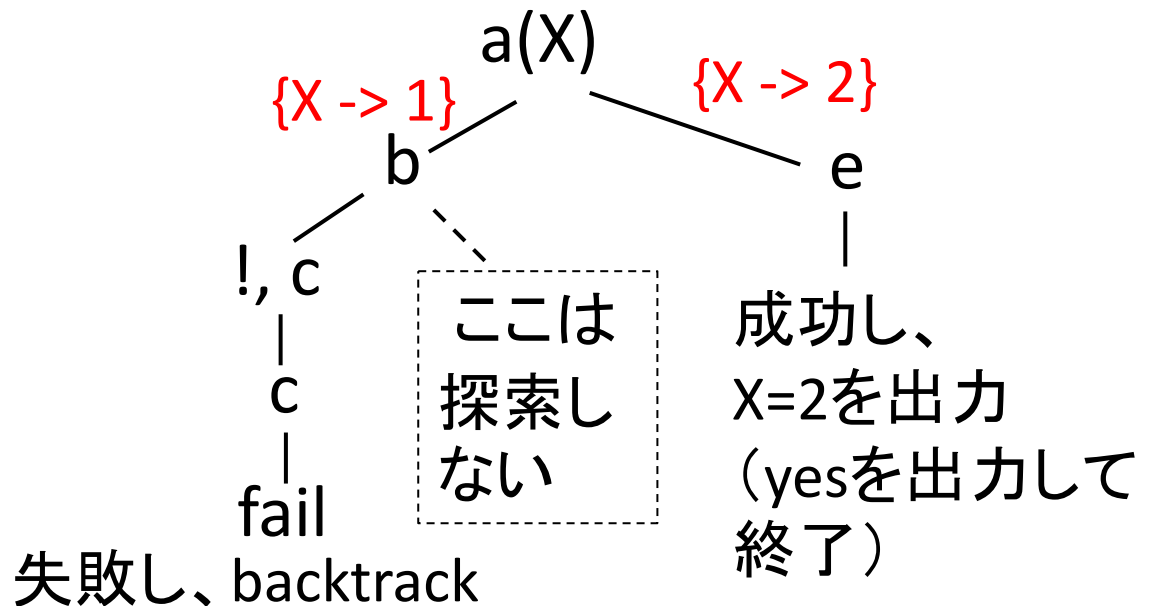
- 探索する部分を削減する効果を持ち、計算量が減少
- 純粋な論理式の意味からははずれる

$B :- C_1, \dots, C_{j-1}, !, C_{j+1}, \dots, C_k$

!自体は常に成功し、backtrackで!に戻ってきた際に、述語 B は失敗するという副作用を持つ。

$a(1) :- b.$
 $a(2) :- e.$
 $b :- !, c.$
 $b :- d.$
 $c :- \text{fail}.$
 $d.$
 $e.$

?- $a(X).$
 $X=2$
 yes



Cutの例

```
mem(K, node(K,_,_)).
```

```
mem(K, node(N,S,_)) :- K<N, mem(K,S).
```

```
mem(K, node(N,_,T)) :- K>N, mem(K,T).
```

この定義では、3つの規則が相互に重なり合わない
ので、以下のようにcutを入れると効率が良くなる。

```
mem(K, node(K,_,_)).
```

```
mem(K, node(N,S,_)) :- K<N, !, mem(K,S).
```

```
mem(K, node(N,_,T)) :- K>N, mem(K,T).
```

この場合は、Prolog探索木の中で解に到達しない部分だけを探索範囲から除外している。このようなcutをgreen cutという。それ以外のcutはred cutという。

not演算子

[Prologのnot演算子]

$\text{Y}+(Y) :- Y, !, \text{fail}.$

$\text{Y}+(_).$

?- $X=2, \text{Y}+(X=1).$

$X=2$

yes

?- $\text{Y}+(X=1), X=2.$

no

$X=2, \text{Y}+(X=1)$

$\{X \rightarrow 2\} \mid$

$\text{Y}+(2=1)$
 $\{Y \rightarrow 2=1\}$

$2=1, !, \text{fail}$ 成功し、 $X=2$ を出力
 $2=1$ が失敗し、（*yes*を出力して終了）
backtrack

$\text{Y}+(X=1), X=2$

$\{Y \rightarrow X=1\}$

$X=1, !, \text{fail}, X=2$

$\{X \rightarrow 1\} \mid$

$!, \text{fail}, 1=2$

$\text{fail}, 1=2$

fail が失敗し、backtrack

ここは
探索し
ない

（*no*を出力して
終了）